

10 ROS2 인터페이스

2101080 정진용

1.

- 메시지, 토픽, 서비스, 액션, 인터페이스 용어를 명확히 구분하여 설명하라.

(Answer)

인터페이스 (Interface)

인터페이스는 ROS 2 통신 시스템의 가장 근간이 되는 데이터 구조 설계도입니다. 특정 프로그래밍 언어에 구애받지 않는 중립적인 방식으로 통신에 사용될 데이터의 형식과 구조를 정의하는 규칙 그 자체를 의미합니다. .msg, .srv, .action이라는 세 가지 유형의 텍스트 파일을 사용하여 데이터 필드의 타입과 이름을 명시하는데, ROS 2 빌드 시스템은 이 파일을 기반으로 C++나 Python과 같은 각 언어에서 사용할 수 있는 코드를 자동으로 생성해 줍니다. 이 덕분에 서로 다른 언어로 작성된 노드들이 동일한 인터페이스 정의를 공유함으로써 원활하게 데이터를 교환할 수 있는 언어 간 상호 운용성이 보장됩니다.

메시지 (Message)

메시지는 ROS 2에서 데이터를 주고받는 가장 기본적인 단위이자 인터페이스의 한 종류입니다. .msg 파일에 정의되며, int32, string, float64 등과 같은 기본 데이터 타입과 다른 메시지 타입을 조합하여 복잡한 데이터 구조를 만들 수 있습니다. 이렇게 정의된 메시지는 토픽 통신을 통해 직접적으로 발행(Publish)되거나 구독(Subscribe)되며, 더 복잡한 통신 방식인 서비스나 액션에서도 요청(Request), 응답(Response), 피드백(Feedback) 등의 데이터를 구성하는 핵심 요소로 사용됩니다. 즉, 메시지는 모든 ROS 2 통신의 기초가 되는 '데이터 패킷'의 설계도라고 할 수 있습니다.

토픽 (Topic)

토픽은 일대다(one-to-many) 비동기 통신을 위한 메커니즘으로, 발행/구독(Publish/Subscribe) 모델을 사용합니다. 특정 이름(예: /turtle1/pose)을 가진 채널처럼 작동하며, 발행자(Publisher) 노드는 이 채널에 지속적으로 메시지를 보내고, 해당 토픽을 구독(Subscriber)하는 모든 노드는 그 메시지를 비동기적으로 수신합니다. 발행자는 누가 메시지를 받는지 알 필요가 없으며, 구독자 역시 누가 메시지를 보내는지 알 필요 없이 오직 토픽 이름과 메시지 타입만 일치하면 통신이 이루어집니다. 이러한 특성 때문에 센서 데이터 스트리밍이나 로봇의 상태를 계속해서 방송하는 등 연속적인 정보를 전달하는 데 매우 적합합니다.

서비스 (Service)

서비스는 일대일(one-to-one) 동기 통신을 위한 메커니즘으로, 원격 프로시저 호출(RPC)과 유사한 요청/응답(Request/Response) 모델을 따릅니다. 클라이언트(Client) 노드가 특정 서버(Server) 노드에 작

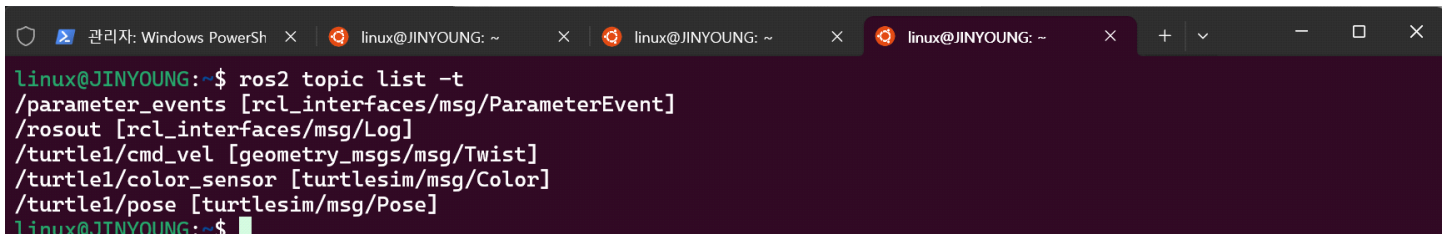
업을 요청하는 메시지를 보내면, 클라이언트는 서버가 작업을 완료하고 응답 메시지를 보내줄 때까지 대기합니다. 이 통신은 특정 계산을 수행하거나, 매개변수를 변경하거나, 특정 동작을 한 번만 수행하도록 명령하는 등, 완료까지 시간이 오래 걸리지 않고 반드시 결과 확인이 필요한 작업에 사용됩니다. 서비스 인터페이스는 .srv 파일에 정의되며, 요청과 응답의 데이터 구조가 --- 구분자를 기준으로 함께 명시됩니다.

액션 (Action)

액션은 장시간이 소요되는 비동기 작업을 위한 통신 메커니즘으로, 서비스의 한계를 보완하기 위해 설계되었습니다. 액션 클라이언트가 액션 서버에게 목표(Goal)를 보내면, 서버는 목표를 수행하는 동안 주기적으로 현재 진행 상황을 담은 피드백(Feedback)을 클라이언트에게 보낼 수 있습니다. 작업이 모두 완료되면 최종 결과(Result)를 전송합니다. 서비스와 달리 클라이언트는 응답을 기다리며 멈춰있지 않아도 되며, 작업 중간에 목표를 취소할 수도 있습니다. 이러한 특징 덕분에 로봇을 특정 위치로 이동시키는 내비게이션이나 복잡한 팔 동작과 같이, 실행 시간이 길고 중간 과정의 확인이 필요한 임무를 수행하는 데 매우 유용합니다.

2.

- ros2 명령어를 이용하여 turtlesim과 teleop_turtle노드를 각각 실행하고 현재 실행중인 토픽메시지와 메시지 인터페이스를 출력하시오.
- (Answer)



```
linux@JINYOUNG:~$ ros2 topic list -t
/parameter_events [rcl_interfaces/msg/ParameterEvent]
/rosout [rcl_interfaces/msg/Log]
/turtle1/cmd_vel [geometry_msgs/msg/Twist]
/turtle1/color_sensor [turtlesim/msg/Color]
/turtle1/pose [turtlesim/msg/Pose]
linux@JINYOUNG:~$
```

3.

- 앞에서 출력한 메시지 인터페이스의 정의를 ros2 명령어를 이용하여 각각 출력하시오.

(Answer)

/parameter_events [rcl_interfaces/msg/ParameterEvent]

```
linux@JINYOUNG:~$ ros2 interface show rcl_interfaces/msg/ParameterEvent
# This message contains a parameter event.
# Because the parameter event was an atomic update, a specific parameter name
# can only be in one of the three sets.

# The time stamp when this parameter event occurred.
builtin_interfaces/Time stamp

# Fully qualified ROS path to node.
string node

# New parameters that have been set for this node.
Parameter[] new_parameters

# Parameters that have been changed during this event.
Parameter[] changed_parameters

# Parameters that have been deleted during this event.
Parameter[] deleted_parameters
linux@JINYOUNG:~$ █
```

rcl_interfaces/msg/Log

```
linux@JINYOUNG:~$ ros2 interface show rcl_interfaces/msg/Log
##
## Severity level constants
##
## These logging levels follow the Python Standard
## https://docs.python.org/3/library/logging.html#logging-levels
## And are implemented in rcutils as well
## https://github.com/ros2/rcutils/blob/35f29850064e0c33a4063cbc947ebbfeda11dba/include/rcutils/logging.h#L164-L172
## This leaves space for other standard logging levels to be inserted in the middle in the future,
## as well as custom user defined levels.
## Since there are several other logging enumeration standard for different implementations,
## other logging implementations may need to provide level mappings to match their internal implementations.
##

# Debug is for pedantic information, which is useful when debugging issues.
byte DEBUG=10

# Info is the standard informational level and is used to report expected
# information.
byte INFO=20

# Warning is for information that may potentially cause issues or possibly unexpected
# behavior.
byte WARN=30

# Error is for information that this node cannot resolve.
byte ERROR=40

# Information about a impending node shutdown.
byte FATAL=50

##
## Fields
##
# Timestamp when this message was generated by the node.
builtin_interfaces/Time stamp

# Corresponding log level, see above definitions.
uint8 level

# The name representing the logger this message came from.
string name

# The full log message.
string msg

# The file the message came from.
string file

# The function the message came from.
string function

# The line in the file the message came from.
uint32 line
linux@JINYOUNG:~$
```

geometry_msgs/msg/Twist

```
linux@JINYOUNG:~$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3 linear
Vector3 angular
linux@JINYOUNG:~$
```

turtlesim/msg/Color

```
linux@JINYOUNG:~$ ros2 interface show turtlesim/msg/Color
uint8 r
uint8 g
uint8 b
linux@JINYOUNG:~$
```

turtlesim/msg/Pose

```
Linux@JINYOUNG:~$ ros2 interface show turtlesim/msg/Pose
float32 x
float32 y
float32 theta

float32 linear_velocity
float32 angular_velocity
Linux@JINYOUNG:~$
```